



Amazon VPC — Complete Notes

Virtual Private Cloud · Your isolated network on AWS

AWS VPC

Service Networking

Scope Regional

Network Isolated

A reader-friendly, all-in-one reference for Amazon VPC.

Table of Contents

1. What is a VPC
2. Core Components
3. CIDR & IP Addressing
4. Subnets (Public vs Private)
5. Route Tables
6. Gateways (IGW, NAT, etc.)
7. Security Groups vs NACLs
8. VPC Connectivity Options
9. VPC Endpoints & PrivateLink
10. DNS in a VPC
11. VPC Flow Logs & Monitoring
12. Common CLI Commands
13. Key Limits & Numbers
14. Design Best Practices
15. Quick Mental Model

1. What is a VPC

Amazon VPC lets you provision a **logically isolated virtual network** within AWS where you launch resources (EC2, RDS, Lambda, etc.). You fully control the IP address range, subnets, route tables, gateways, and firewall rules — like running your own data-center network, but software-defined.

 **Mental model:** a VPC is your **private slice of the AWS cloud**. It's **regional** (spans all AZs in one region); subnets live in a single AZ.

Why it matters: isolation, segmentation (public vs private tiers), fine-grained traffic control, and secure connectivity to on-prem and other VPCs.

2. Core Components

Component	Role
VPC	The network container with a CIDR block; regional.
Subnet	A slice of the VPC CIDR, bound to one AZ.
Route Table	Rules deciding where subnet traffic goes.
Internet Gateway (IGW)	Allows internet access for public subnets.
NAT Gateway	Lets private subnets reach the internet outbound only.
Security Group	Stateful firewall at the instance/ENI level.
Network ACL	Stateless firewall at the subnet level.
ENI	Virtual network interface attached to resources.
Elastic IP	Static public IPv4 you own.
VPC Endpoint	Private access to AWS services without the internet.
Peering / TGW / VPN / DX	Ways to connect VPCs and on-prem networks.

3. CIDR & IP Addressing

- A VPC is defined by a **CIDR block**, e.g. `10.0.0.0/16` (65,536 addresses).
- Allowed VPC CIDR size: **/16 (max) to /28 (min)**. Use **private ranges** (RFC 1918): `10.0.0.0/8` , `172.16.0.0/12` , `192.168.0.0/16` .
- Subnets carve the VPC CIDR into smaller blocks, e.g. `10.0.1.0/24` (256 addresses).
- **AWS reserves 5 IPs per subnet** (first 4 + last 1): network, VPC router, DNS, future use, broadcast. So a `/24` gives **251 usable** IPs.
- You can add **secondary CIDR blocks** to a VPC and enable **IPv6** (AWS assigns a `/56` , subnets get `/64`).

⚠ Plan CIDRs so VPCs that may peer or connect to on-prem **don't overlap** — overlapping ranges can't be routed between.

4. 🚧 Subnets (Public vs Private)

	Public Subnet	Private Subnet
Route to IGW	✅ Yes	❌ No
Internet inbound	Possible (with public IP)	No
Internet outbound	Direct via IGW	Via NAT Gateway only
Typical use	Load balancers, bastions, NAT GW	App servers, databases

- A subnet is **public** when its route table has a route `0.0.0.0/0 → IGW`.
- Spread subnets across **multiple AZs** for high availability.
- Common 3-tier layout: public (web/LB) · private-app · private-data, each replicated per AZ.

5. 🌐 Route Tables

- Each subnet associates with **exactly one** route table; a route table can serve many subnets.
- The **main route table** is the default; create **custom** ones per tier.
- Routes map a **destination CIDR** → **target** (IGW, NAT GW, ENI, peering, TGW, endpoint, etc.).
- The **local route** (`VPC CIDR → local`) is always present and enables intra-VPC routing — it can't be removed.
- **Most specific match wins** (longest prefix).

Destination	Target	
10.0.0.0/16	local	(intra-VPC, implicit)
0.0.0.0/0	igw-xxxx	(public subnet → internet)
0.0.0.0/0	nat-xxxx	(private subnet → NAT)

6. 🏠 Gateways (IGW, NAT, etc.)

Gateway	Direction	Notes
Internet Gateway (IGW)	In + out	One per VPC; horizontally scaled, no bandwidth limit. Required for public internet.
NAT Gateway	Outbound only	Managed, per-AZ, needs an EIP. Lets private subnets pull updates/reach internet. Charged hourly + per GB.
NAT Instance	Outbound only	Legacy DIY (an EC2 doing NAT). Cheaper but you manage it.
Egress-Only IGW	Outbound only (IPv6)	The IPv6 equivalent of a NAT gateway.
Virtual Private Gateway (VGW)	VPN	AWS side of a Site-to-Site VPN.
Transit Gateway (TGW)	Hub	Connects many VPCs + on-prem at scale.

🔑 NAT Gateways cost per hour **and** per GB processed — a common surprise bill. Use **VPC Gateway Endpoints** for S3/DynamoDB to avoid NAT data charges.

7. 🛡️ Security Groups vs NACLs

	Security Group	Network ACL
Level	Instance / ENI	Subnet
State	Stateful (return traffic auto-allowed)	Stateless (allow both directions)
Rules	Allow only	Allow and Deny
Evaluation	All rules together	Numbered order, first match wins
Default	Deny inbound / allow outbound	Default allows all; custom denies all

✅ Use **Security Groups** as your primary control (reference other SGs as sources for tier-to-tier rules). Use **NACLs** for coarse subnet-wide allow/deny (e.g. block a bad IP range).

8. 🔗 VPC Connectivity Options

Option	Connects	Notes
VPC Peering	VPC ↔ VPC (1:1)	Non-transitive; simple; can be cross-region/cross-account. CIDRs must not overlap.
Transit Gateway (TGW)	Many VPCs + VPN/DX (hub-spoke)	Scales to thousands of VPCs; transitive routing.
Site-to-Site VPN	On-prem ↔ VPC over internet	Encrypted IPsec tunnels via VGW or TGW.
Direct Connect (DX)	On-prem ↔ AWS dedicated line	Private, consistent bandwidth/latency; not over internet.
PrivateLink / Endpoints	VPC ↔ AWS or partner service	Private, no IGW/NAT (see next section).

Peering is **not transitive**: if A↔B and B↔C peer, A can't reach C through B. Use a **Transit Gateway** for hub-and-spoke at scale.

9. 🚫 VPC Endpoints & PrivateLink

Reach AWS services **privately**, without an IGW, NAT, or public IPs.

Endpoint type	How	Services
Gateway Endpoint	A route-table target	S3 and DynamoDB only . Free.
Interface Endpoint (PrivateLink)	An ENI with a private IP in your subnet	Most AWS services + partner/SaaS + your own services. Hourly + per-GB cost.

Benefits: traffic stays on the AWS network (more secure), avoids NAT data charges (Gateway Endpoint), and enables private SaaS connectivity (PrivateLink).

10. 🧭 DNS in a VPC

- AWS provides DNS at the **VPC base +2 address** (e.g. `10.0.0.2`) — the **Route 53 Resolver**.
- Two VPC attributes: `enableDnsSupport` (resolution on) and `enableDnsHostnames` (assign public DNS names).
- **Route 53 Private Hosted Zones** give internal domain names resolvable inside the VPC.
- **Resolver endpoints** (inbound/outbound) bridge DNS between on-prem and VPC.

11. 🇮🇹 VPC Flow Logs & Monitoring

- **Flow Logs** capture IP traffic metadata (src/dst, ports, bytes, ACCEPT/REJECT) at VPC, subnet, or ENI level.
- Destinations: **CloudWatch Logs**, **S3**, or **Kinesis Firehose**. Great for troubleshooting connectivity and security forensics.
- Flow Logs do **not** capture packet contents (use traffic mirroring for that).
- **VPC Reachability Analyzer** and **Network Access Analyzer** help validate/troubleshoot paths and exposure.

12. 📄 Common CLI Commands

```
aws ec2 create-vpc --cidr-block 10.0.0.0/16
aws ec2 create-subnet --vpc-id vpc-123 --cidr-block 10.0.1.0/24 --availability-zone us-east-1a
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway --vpc-id vpc-123 --internet-gateway-id igw-123
aws ec2 create-route-table --vpc-id vpc-123
aws ec2 create-route --route-table-id rtb-123 --destination-cidr-block 0.0.0.0/0 --gateway-id igw-123
aws ec2 create-nat-gateway --subnet-id subnet-123 --allocation-id eipalloc-123
aws ec2 describe-vpcs
aws ec2 create-flow-logs --resource-type VPC --resource-ids vpc-123 \
  --traffic-type ALL --log-destination-type cloud-watch-logs --log-group-name vpc-flow
```

13. 📌 Key Limits & Numbers

Item	Value
VPCs per region	5 (default, adjustable)
VPC CIDR size	/16 (max) to /28 (min)
Secondary CIDRs per VPC	up to 5 (adjustable)
Subnets per VPC	200 (default)
Reserved IPs per subnet	5 (first 4 + last 1)
Route tables per VPC	200
Routes per route table	50 (non-propagated)
Security groups per ENI	5 (up to 16)
Rules per security group	60 in + 60 out
Internet Gateways	1 per VPC

14. 🧱 Design Best Practices

- **Plan non-overlapping CIDRs** across all VPCs and on-prem from day one.
 - **Multi-AZ subnets** for every tier → high availability.
 - **Public/private separation**: only LBs, bastions, and NAT in public subnets; everything else private.
 - **Least privilege** Security Groups; reference SGs, not raw IPs, for tier-to-tier traffic.
 - Use **Gateway Endpoints** (S3/DynamoDB) and **Interface Endpoints** to cut NAT cost and stay private.
 - Centralize multi-VPC connectivity with a **Transit Gateway**, not a mesh of peerings.
 - Turn on **Flow Logs** for security and troubleshooting.
-

15. 🧠 Quick Mental Model

- **VPC = your private network in AWS**, defined by a CIDR, regional in scope.
 - **Subnets** split it per AZ; **public** = route to IGW, **private** = route to NAT.
 - **Route tables** decide where traffic goes; **most specific route wins**.
 - **Security Groups** (stateful, instance) + **NACLs** (stateless, subnet) filter traffic.
 - Connect out: **IGW** (public), **NAT** (private outbound), **VPN/DX** (on-prem), **Peering/TGW** (VPC-to-VPC).
 - Reach AWS services privately with **Endpoints / PrivateLink**.
 - **Don't overlap CIDRs**, spread across AZs, keep data tiers private, and watch **NAT data costs**.
-

📝 Notes compiled as a quick reference for Amazon VPC.